Chapter 5

Shared-Memory Programming

5.1 Learning Objectives

Students should be able to classify shared and private state, set up a Pthreads compiler and run a first example, distinguish a data race from a logical race, identify a critical-section invariant, select a mutex or atomic operation, explain release/acquire publication, and reason about contention. This chapter prepares for the fuller Pthreads programs of Chapter 6 and OpenMP in Chapter 7.

5.2 Shared Address Space and Ownership

Threads in one process share its virtual address space: global variables, heap objects, and arrays are reachable by all threads given a suitable pointer. Each thread has its own registers and stack. A stack object can nevertheless be shared if its address is passed to another thread while the object remains alive. Thus, the question is not where an object is allocated but which threads can reach it and whether they write it.

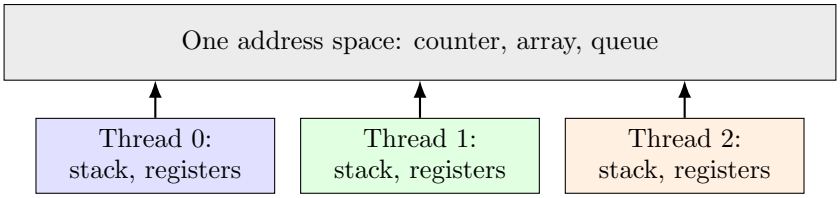


Figure 5.1: Threads have private execution state but may access the same objects. Arrows do not imply synchronization.

MPI ranks in Chapters 3 and 4 use separate address spaces and exchange explicit messages. Shared-memory threads can communicate by reading and writing the same location, but a shared address does not make an operation indivisible or make a new value safely visible.

Table 5.1: Classifying objects by access

Object	Possible access	Question to ask
Global counter	Many writers	What protects increments?
Heap array, disjoint ranges	One writer per element	Are index ranges truly disjoint?
Worker loop index	Usually one worker	Was its address shared?
Queue and length	Producers and consumers	Do the fields change as one invariant?

5.3 Race Conditions: A Worked Interleaving

Suppose a counter starts at 10. The expression `counter++` involves a read, addition, and write. Two threads can each calculate 11 and both store it, losing one increment.

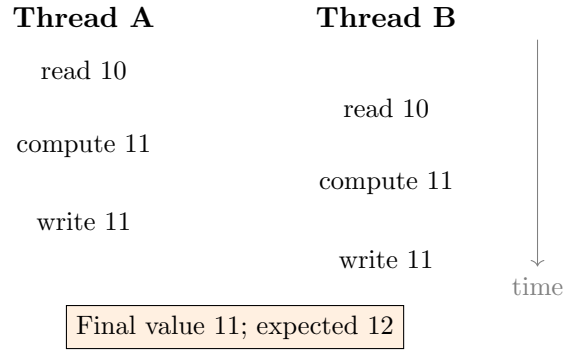


Figure 5.2: An explanatory lost-update interleaving.

A *data race* occurs when two threads access the same memory location, at least one writes, and the accesses lack the synchronization required by the language memory model. In C, a data race on an ordinary object gives undefined behavior. The diagram illustrates the hazard; it does not promise that a racy C program will print 11. A test that prints 12 once is no proof of correctness. Two workers writing different array elements do not race on those elements, though they may race on a shared index used to select them.

5.3.1 A wrong result without a data race

Atomic access removes a data race on that object, but several atomic operations can still form an incorrect algorithm. Suppose one ticket remains. Two sellers each read an atomic `tickets` value of 1, decide a sale is allowed, and then atomically subtract 1. The individual reads and subtractions are defined, yet the final value is -1 and two tickets were sold. The mistake is a *logical race*: correctness depends on which operations occur between the check and the update. A mutex covering both steps or a compare-and-exchange loop can preserve the invariant. This distinction matters because replacing every ordinary variable with an atomic variable is not a general repair.

Table 5.2: Atomic operations can still violate a compound rule

Step	Seller A	Seller B	Tickets
1	Reads 1; sale allowed		1
2		Reads 1; sale allowed	1
3	Atomically subtracts 1		0
4		Atomically subtracts 1	-1

5.4 Critical Sections and Invariants

A critical section protects a shared relationship that must be true before and after each operation. For a balance B, one possible invariant is $B \geq 0$. Checking sufficient funds and subtracting must be one operation:

Listing 5.1: Protect the complete check-and-update operation

```
pthread_mutex_lock(&account_lock);
if (balance >= withdrawal) {
    balance -= withdrawal;
    approved = 1;
}
pthread_mutex_unlock(&account_lock);
```

If $B = 100$ and two workers request 80, a lock around only the subtraction lets both pass a stale check of 100. The lock must cover every access that participates in the invariant, including reads that require a consistent snapshot. Keep unrelated work outside the critical section so other workers wait less. When several locks are needed, acquire them in a consistent order to prevent circular waiting; Chapter 12 examines deadlock in detail.

5.5 Getting Started with Pthreads

Pthreads is the POSIX threads interface used by C programs on Linux and macOS. The examples in this chapter require a C11 compiler, the `pthread.h` header, and the threading build option `-pthread`. The option belongs in the compile command because it selects the required thread-aware compilation and linking behavior. The program below is the first build exercise; Chapter 6 develops the thread API in detail.

5.5.1 Linux and macOS

On Ubuntu or Debian, install the C toolchain in a terminal:

Listing 5.2: Ubuntu or Debian toolchain setup

```
sudo apt update
sudo apt install build-essential
cc --version
```

On macOS, install Apple's Command Line Tools if `cc --version` does not find a compiler:

Listing 5.3: macOS toolchain setup

```
xcode-select --install
cc --version
```

Installation instructions are maintained by the Ubuntu compiler guide and Apple Command Line Tools guide. The macOS command opens an installer dialog; run the example after that installation finishes.

5.5.2 Windows: WSL 2 with Ubuntu

WSL runs a Linux environment on Windows. For students following POSIX-oriented course examples, this is the most direct route. On a supported Windows 10 or Windows 11 computer, open *PowerShell as Administrator* and run:

Listing 5.4: Install Ubuntu through WSL from Windows PowerShell

```
wsl --install -d Ubuntu
```

Restart if prompted. Launch Ubuntu, complete its first-run account setup, and then enter the following commands *in the Ubuntu terminal*, not in PowerShell:

Listing 5.5: Install the C compiler inside Ubuntu on WSL

```
sudo apt update
sudo apt install build-essential
cc --version
```

From PowerShell, `wsl -l -v` lists the installed distribution and WSL version. If WSL is already installed, use the Microsoft WSL installation guide to select or add Ubuntu. WSL builds a Linux executable; run it inside the Ubuntu terminal.

Put a copy of this project in your Ubuntu home directory and use `cd` to enter its root before building. Windows files on drive C are also accessible under `/mnt/c`, although Linux-based build work is usually faster when the project is stored in the WSL file system. See Microsoft's WSL file-system guide.

5.5.3 Windows: native build with MSYS2 UCRT64

An alternative is a native Windows build. Install MSYS2 from its official getting-started page, open the *MSYS2 UCRT64* terminal, update its packages as instructed by the installer, and install the UCRT64 GCC package:

Listing 5.6: Native Windows compiler setup in the MSYS2 UCRT64 terminal

```
pacman -Syu
pacman -S mingw-w64-ucrt-x86_64-gcc
gcc --version
```

If the update asks you to close the terminal, reopen *MSYS2 UCRT64*, run `pacman -Syu` again, and finish updating before installing GCC. Its GCC package includes the Winpthreads dependency. Use the UCRT64 terminal for compiling and running; the ordinary MSYS shell selects a different toolchain. The MSYS2 environment guide explains this distinction.

Keep the project on a Windows drive and use `cd` in the UCRT64 terminal to reach its root. In either Windows route, run `ls examples/chapter5/counter_modes.c` before compiling: it should print that filename. If it does not, you are in the wrong directory or have not copied the project.

5.5.4 First build and expected result

From the root of this book's project, after reaching the complete source file below, build and run the example. On Linux, macOS, or Ubuntu under WSL:

Listing 5.7: Build and run from the project root on POSIX systems

```
cc -std=c11 -O2 -Wall -Wextra -pthread \
  examples/chapter5/counter_modes.c -o counter_modes
./counter_modes
```

In MSYS2 UCRT64, use `gcc` and a Windows executable name:

Listing 5.8: Build and run from the project root in MSYS2 UCRT64

```
gcc -std=c11 -O2 -Wall -Wextra -pthread \
  examples/chapter5/counter_modes.c -o counter_modes.exe
./counter_modes.exe
```

Both commands should print `mutex=200000 atomic=200000 expected=200000`. If the compiler cannot find `pthread.h`, check which terminal and compiler you opened. If linking reports

missing `pthread_...` symbols, check that `-pthread` appears in the build command. If the source path is missing, change to the project root before compiling. The command and expected output form a small check that the environment is ready; they do not establish that every later Pthreads program is correct.

5.6 Atomics and a Complete Counter Program

An atomic read-modify-write changes one object indivisibly under the C memory model. It suits an independent event count. It does not make a compound change to several ordinary variables transactional. The complete program below uses both a mutex-protected count and a C11 atomic count. The first listing defines the shared state and worker routine; the second provides the main routine. Together they form `examples/chapter5/counter_modes.c`. Chapter 6 explains thread creation and joining; the point here is that each shared update has an appropriate protection rule.

Listing 5.9: Shared counters and worker routine

```c
/* Chapter 5: two correct ways to count concurrent events. */
#include <pthread.h>
#include <stdatomic.h>
#include <stdio.h>
#include <stdlib.h>

enum { WORKERS = 2, ITERATIONS = 100000 };
static long locked_count = 0;
static atomic_long atomic_count = ATOMIC_VAR_INIT(0);
static pthread_mutex_t count_lock = PTHREAD_MUTEX_INITIALIZER;

static void *worker(void *unused) {
    (void)unused;
    for (int i = 0; i < ITERATIONS; ++i) {
        if (pthread_mutex_lock(&count_lock) != 0) abort();
        ++locked_count;
        if (pthread_mutex_unlock(&count_lock) != 0) abort();
        atomic_fetch_add_explicit(&atomic_count, 1, memory_order_relaxed);
    }
    return NULL;
}
```

Listing 5.10: Main routine for the two-counter program

```
int main(void) {
    pthread_t threads[WORKERS];
    for (int i = 0; i < WORKERS; ++i)
        if (pthread_create(&threads[i], NULL, worker, NULL) != 0) abort();
    for (int i = 0; i < WORKERS; ++i)
        if (pthread_join(threads[i], NULL) != 0) abort();
    printf("mutex=%ld atomic=%ld expected=%ld\n",
           locked_count,
           atomic_load_explicit(&atomic_count, memory_order_relaxed),
           (long)WORKERS * ITERATIONS);
    if (pthread_mutex_destroy(&count_lock) != 0) abort();
    return 0;
}
```

Use the build command for your environment from the preceding quickstart. Two workers make 100,000 updates each, so both counts must be 200,000. The atomic uses `memory_order_relaxed` because this count publishes no other data. The mutex provides exclusion and ordering for the ordinary counter.

5.6.1 Reading the program

The two file-scope counters are reachable by both workers. `locked_count` is an ordinary `long`; every increment is enclosed by locking and unlocking `count_lock`. While one worker owns the lock, the other cannot enter that region, so no increments overlap. `atomic_count` is an `atomic_long`; `atomic_fetch_add_explicit` performs each increment as one atomic read-modify-write. The return value of fetch-add is unused because this example needs only the final count.

The first loop in `main` starts exactly two workers. The second loop joins both before printing; by then no worker can modify either counter. Multiplying `WORKERS` by `ITERATIONS` gives an independent expected value. On a successful run the program prints:

Listing 5.11: Expected result for the supplied constants

```
mutex=200000 atomic=200000 expected=200000
```

Changing the scheduling order cannot change the two counts. The program deliberately performs both update methods in each iteration, so its runtime is *not* a fair benchmark of mutexes against atomics. A performance experiment would measure separate versions, repeat runs, and report the machine and workload.

Table 5.3: Match the mechanism to the invariant

Operation	Mechanism	Reason
Increment independent count	Atomic fetch-add	One object changes indivisibly.
Check and withdraw funds	Mutex	Check and update must stay together.
Change queue entry and length	Mutex	Several fields must agree.
Read fixed lookup table	No per-read lock	No writer changes it during reads.

5.7 Memory Visibility and Publication

Compilers and processors may reorder operations unless a program establishes the required synchronization. An ordinary readiness flag accessed concurrently by producer and consumer is itself a data race. The following C11 pattern publishes an ordinary payload through an atomic flag:

Listing 5.12: Release/acquire publication; the blocks run in separate threads

```c
#include <stdatomic.h>
int payload = 0;
atomic_int ready = ATOMIC_VAR_INIT(0);

/* Producer */
payload = 42;
atomic_store_explicit(&ready, 1, memory_order_release);

/* Consumer */
while (atomic_load_explicit(&ready, memory_order_acquire) == 0) {
    /* wait; blocking coordination may be preferable */
}
int answer = payload; // 42
```

When the acquire load reads the released value, the payload write *happens before* the consumer reads the payload. This example assumes one publication and no later unsynchronized payload writer. Relaxed operations would keep the flag atomic but would not establish that ordering for the payload. Unlocking a mutex and later locking that same mutex also establishes an ordering boundary.

Table 5.4: What the synchronization in this chapter guarantees

Operation	Guarantee used here	Appropriate example
Relaxed atomic fetch-add	Indivisible change to one atomic object	Independent counter
Release store observed by acquire load	Earlier writes become ordered before later reads	Publish completed payload
Mutex unlock followed by lock of same mutex	Protected operations are exclusive and ordered	Balance or queue invariant

The C qualifier `volatile` does not replace any of these mechanisms. It does not make an increment atomic or establish a happens-before relationship between threads. Device registers and other special uses of `volatile` are separate topics; ordinary shared program data needs the synchronization rules above.

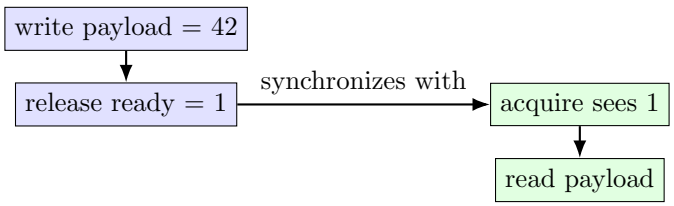


Figure 5.3: The release/acquire handoff publishes an earlier ordinary write.

5.8 Coordination Beyond Mutual Exclusion

A mutex answers who may modify protected state now. A condition variable lets a worker sleep until a protected predicate may have changed; after waking, the worker must test it again. A barrier separates phases. A semaphore counts available permits. A read-write lock allows concurrent readers but excludes writers. These solve different problems: a barrier does not protect a counter, and a mutex alone does not notify a consumer that work has arrived.

For a bounded queue of capacity K and length n, maintain $0 \le n \le K$. A producer inserts only when $n < K$; a consumer removes only when $n > 0$. Queue entries and length change under one mutex. The predicates not-full and not-empty govern waiting. Chapter 6 develops the condition-variable implementation.

For example, let $K = 2$ and start with an empty queue. After inserting items A and B, $n = 2$ and another producer must wait. A consumer removes A, making $n = 1$, so a waiting producer may become eligible to insert C. Waking a producer does not reserve that free slot: another producer might acquire the mutex first. This is why a waiting thread checks the queue predicate again after waking. The invariant $0 \le n \le 2$ holds after each completed operation.

Table 5.5: Synchronization questions

Primitive	Question answered	Example
Mutex	Who owns this invariant?	Account or queue update
Atomic operation	Can one value change indivisibly?	Counter or flag
Condition variable	Has a predicate changed?	Empty/full queue
Barrier	Has everyone finished this phase?	Phase boundary
Semaphore	How many permits remain?	Resource pool

5.9 Contention and False Sharing

Suppose P workers each make M updates. One mutex-protected counter forces all PM updates through one critical section. If each worker instead keeps a private subtotal and the program combines the P results at the end, it needs only about P shared updates. This works when a live total is unnecessary. Chapter 6 implements a parallel reduction based on this principle.

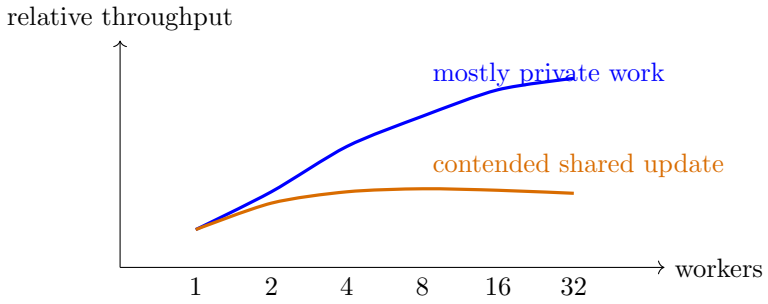


Figure 5.4: Schematic trends, not benchmark data. Serial updates can flatten throughput as workers are added.

Lock contention means workers wait for the same lock. False sharing means workers write different variables on one cache line, causing coherence traffic even without a data race. Chapter 2 explains the hardware mechanism. At program level, use local accumulation or separate

heavily written per-worker values. Padding and alignment should be guided by measurement because they consume memory and cache-line sizes vary.

5.10 Worked Reasoning Example

Four workers each process 250 records. A shared total should end at $4 \times 250 = 1000$. An ordinary unsynchronized increment races even if one run reports 1000. A mutex makes 1000 protected updates correct; an atomic fetch-add makes 1000 atomic updates correct. Private counts of 250, combined after workers finish, need only four partial results. The final-only method cannot provide a live total to other workers.

5.11 Summary

Identify shared locations and invariants before selecting synchronization. A data race on ordinary C state is undefined; an algorithm using only atomics can still violate a compound invariant. A mutex protects such an invariant, an atomic operation handles one indivisible value, and release/acquire ordering publishes earlier data. Correct programs may still scale poorly when shared updates are frequent or per-worker values share a cache line.

5.12 Exercises

1. Starting from 5, show a six-step interleaving of two increments that leaves 6. Why is it only an illustration of a C data race?

2. A queue has entries, head, tail, and length. State two invariants and identify the fields protected by one mutex.

3. Change the counter program to four workers and 50,000 iterations. Predict both outputs.

4. Why is a relaxed flag insufficient to publish an ordinary payload? Give the required store and load orders.

5. For $P = 8$ and $M = 10{,}000$, compare shared-update counts for per-record counting and private subtotals.

6. Two workers update different elements of a counter array. Explain how false sharing can still hurt throughput.

7. Two sellers both read one remaining ticket through an atomic variable, then both atomically subtract one. Is there a data race on the ticket count? Is the sales rule correct? Explain.

8. A programmer replaces an ordinary shared readiness flag with `volatile int`. Does this publish the payload in the C example? Explain.

9. On Windows, state which terminal runs `wsl --install -d Ubuntu`, which terminal runs `sudo apt install build-essential`, and which terminal runs `pacman -S mingw-w64-ucrt-x86_64-gcc`.

5.13 Selected Worked Answers

1. Both read 5, both compute 6, both write 6. These six actions explain a lost update, but racy C has no defined result.
2. $0 \leq \text{length} \leq K$ and head/tail must be valid positions. Protect all fields whose combined state must be consistent.
3. Both counts should be $4 \times 50{,}000 = 200{,}000$.
4. Use a release store and an acquire load that observes the released value. Relaxed flag access does not order the payload.
5. Per-record counting needs 80,000 shared updates; private counting combines eight partial results.
6. The elements are separate locations, so there need not be a data race. If both occupy one cache line, repeated writes can still move that line between cores and reduce throughput.
7. Atomic accesses avoid a data race on the count, but the check and decrement are not one operation. Both sellers can pass the check and sell the same remaining ticket.
8. No. `volatile` does not make the flag atomic or order the payload write before the consumer's read.
9. Use Administrator PowerShell for WSL installation, Ubuntu under WSL for `apt`, and MSYS2 UCRT64 for `pacman`.